Course Project Report
Big Data Course

Taxi Fare Prediction Using Chicago Taxi Trips Data

Marsel Berheev, Makar Egorov, Vladislav Strelkov, Rail Sharipov

Innopolis University
Russia
May 2026

1 Introduction

Modern life is becoming faster every day, and work-life balance often suffers because travelling inside large cities can take a significant amount of time. One of the most convenient solutions for urban mobility is taxi transportation.

In this project, we built a machine learning pipeline for predicting the **fare of a taxi ride**. The task was formulated as a regression problem, where the target variable is the taxi fare and the input features describe trip distance, duration, pickup and dropoff locations, payment type, and temporal information.

1.1 Business Objectives

The main business goal of the project is to predict the **taxi fare** before or during the ride using available trip parameters. Such a model can be useful for:

- estimating the approximate cost of a ride for passengers;
- helping taxi companies improve pricing transparency;
- supporting analytics dashboards for ride demand and revenue analysis.

The model uses the following groups of features:

- pickup and dropoff community areas;
- trip distance and trip duration;
- payment type;
- season, day, weekday, and hour of the ride;

The model is considered successful if it achieves:

- $R^2 > 0.35$ on the test set.

2 Data Description

For this project, we used the open and free-to-use Chicago Taxi Trips 2016 Dataset from Kaggle. The dataset contains a large number of taxi trip records with both numerical and categorical features.

The original dataset includes:

- 19 unique features;
- 58 taxi companies;
- approximately 6.93 thousand unique taxi cars;
- a large number of trip records;

- datetime and geospatial features.

Since the full dataset is very large, we selected four monthly partitions representing different seasons of the year. The final data sample contained approximately **6.9 million records**.

The selected files were:

- `chicago_taxi_trips_2016_01.csv`;
- `chicago_taxi_trips_2016_04.csv`;
- `chicago_taxi_trips_2016_07.csv`;
- `chicago_taxi_trips_2016_10.csv`.

These months were chosen to represent winter, spring, summer, and autumn. All data was ingested using Sqoop and stored in HDFS for distributed processing with Spark.

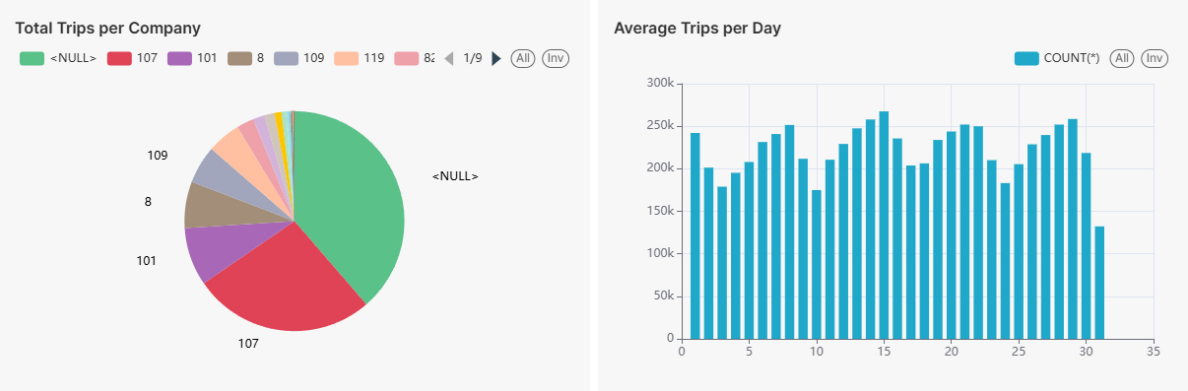

Figure 1: Some dataset Information

3 Data Analysis

Before model training, we performed exploratory data analysis to better understand the structure, quality, and patterns of the data. The analysis produced several important insights.

3.1 Fare Depends on Location

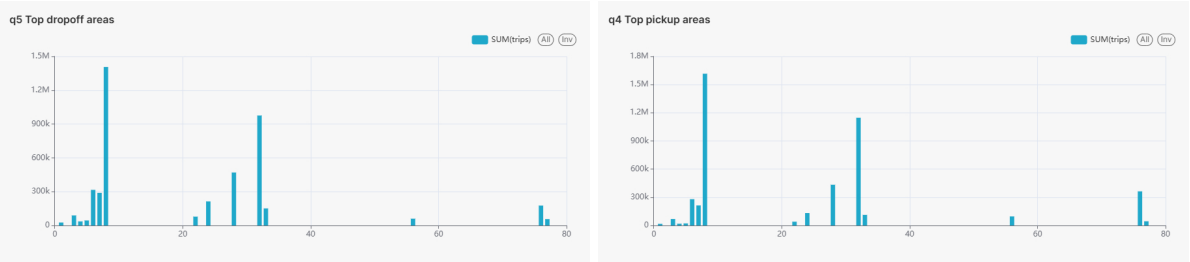

Figure 2: Most Popular Pickups and Dropoffs Positions

The fare distribution differs noticeably between pickup and dropoff community areas. Community area 78 had one of the highest average fares, which may indicate that it is located far from the city center, close to expensive destinations, or connected with longer trips. Community area 8 also had a high average fare and appeared in many records, which suggests that it may be one of the central or high-demand areas of Chicago.

This confirms that location-based features are important for taxi fare prediction.

3.2 Correlation Analysis

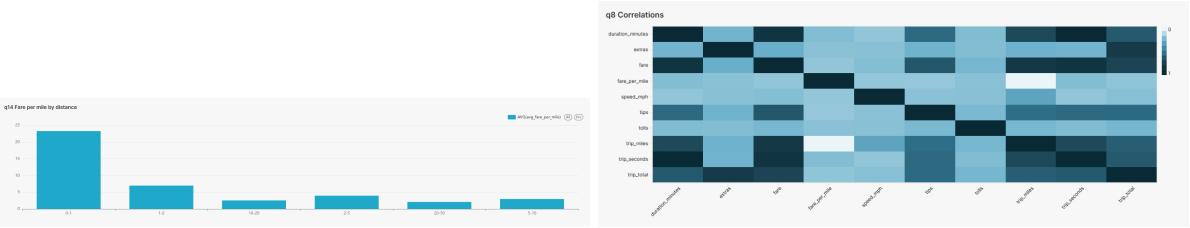

Figure 3: Correlation Matrix and Taarget-Mile Correlations

The correlation matrix showed several expected relationships between numerical variables:

- fare is strongly correlated with `trip_total`, `trip_seconds`, and `trip_miles`;
- trip duration and trip distance are also strongly correlated with each other;
- tips are correlated with fare and trip total.

Since `trip_total` and `tips` contain information that is too close to the target variable, using them could lead to data leakage. Therefore, these columns were not used for model training.

We also observed a negative correlation between `trip_miles` and fare per mile. This may mean that shorter rides are usually more expensive per mile because of fixed starting fees.

3.3 Datetime Analysis

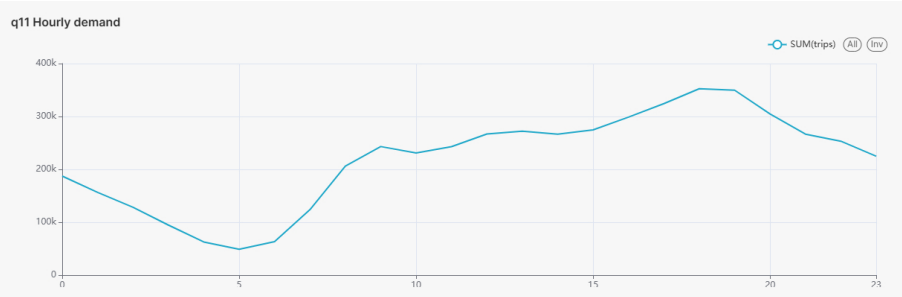

Figure 4: Hourly Demand

Trip demand depends on the time of day. The highest number of trips was observed between 17:00 and 19:00, which is probably caused by the end of the working day, when many people return home. The lowest demand was observed between 04:00 and 06:00, when most people are sleeping and city activity is minimal.

This confirms the importance of temporal features such as hour, weekday, day, and month.

3.4 Missing Values Analysis

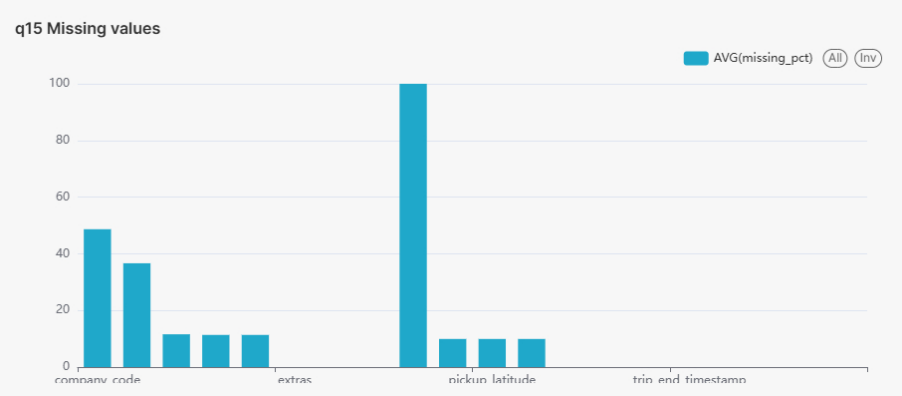

Figure 5: Enter Caption

The dataset contained several columns with a large number of missing values:

- `pickup_census_tract` had 100% missing values and was removed;
- `dropoff_census_tract` had approximately 36% missing values and was removed;
- `company_code` had approximately 48% missing values and did not show a direct relationship with the target variable, so it was also removed.

Removing these columns helped simplify the model and reduce noise in the data.

3.5 Target Distribution

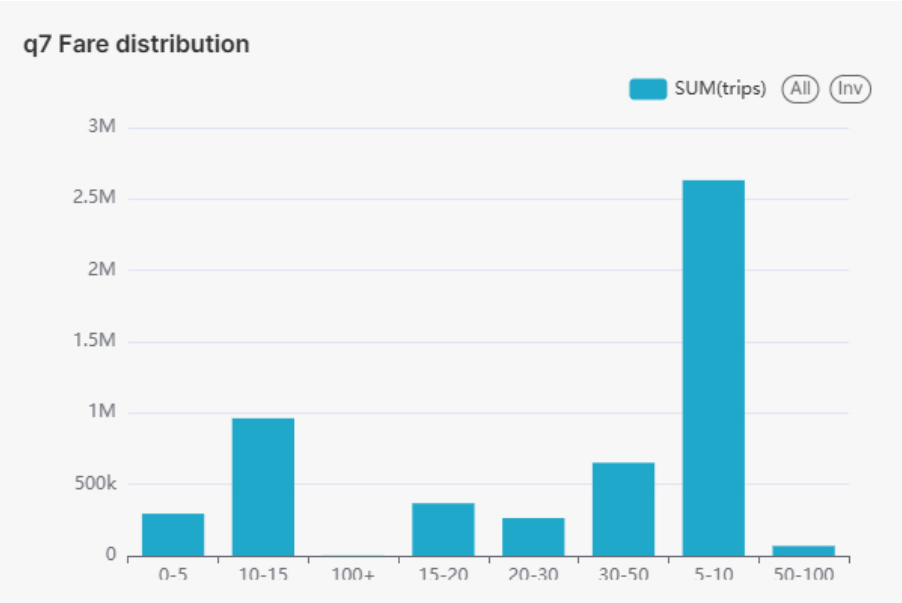

Figure 6: Target Distribution

The target variable, `fare`, had a skewed distribution. Most rides were concentrated in the low and medium fare ranges, especially around the $5–$10 and $10–$25 buckets. This means that the model may be biased toward predicting common fare values.

There were also a small number of high-fare outliers above $100. Such outliers can increase RMSE and make model evaluation more difficult.

3.6 Trip Analysis

Most trips were short, especially in the 0–2 mile range. This means that the dataset is biased toward short urban trips, and the model may predict short rides better than long rides.

Monthly trip counts were mostly uniform, with a small peak in April. Payment type analysis showed that most rides were paid by either cash or credit card. Other payment types were much less frequent, so they were grouped into the `other` category.

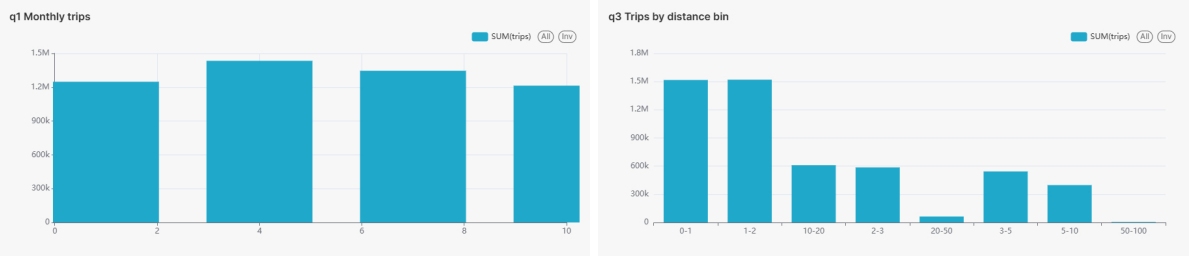

Figure 7: Trips per Month and Distances Bins

3.7 Selected Features

After analysis, we selected the following features for model training:

- `payment_type`;
- `pickup_community_area`;
- `dropoff_community_area`;
- `trip_miles`;
- `trip_seconds`;
- `trip_year`;
- `trip_month`;
- `trip_weekday`;
- `trip_day`;
- `trip_hour`;
- `is_valid_end_timestamp`;
- `is_valid_duration`;
- `is_valid_amounts`;
- `fare` as the target variable.

An example of a processed record is shown below:

Feature	Value
`payment_type`	Cash
`pickup_community_area`	32
`dropoff_community_area`	32
`trip_miles`	0.00
`trip_seconds`	120
`trip_year`	2016
`trip_month`	1
`trip_weekday`	3
`trip_day`	20
`trip_hour`	23
`is_valid_end_timestamp`	true
`is_valid_duration`	true
`fare`	4.75

Table 1: Example of a processed trip record

4 Data Pipeline

The data preprocessing pipeline was implemented using PySpark. The main goal of the pipeline was to transform raw trip records into a clean feature vector suitable for Spark ML models.

4.1 Pipeline Stages

The preprocessing pipeline contained the following stages:

1. **Reading the source table**
   The data was loaded from the Hive/Spark table `team27_projectdb.fact_trips_raw`.

2. **Creating the target column.**
   The `fare` column was cast to double and renamed to `label`, which is the default target column name used by Spark ML models.

3. **Casting numerical features.**
   Numerical columns such as `trip_seconds`, `trip_miles`, and `trip_year` were converted to double type.

4. **Converting boolean features.**
   Boolean validity flags were converted into numerical values: true became 1.0 and false became 0.0.

5. **Processing categorical features.**
   Categorical columns were converted to strings. Missing categorical values were replaced with the value `missing`.

6. **Simplifying payment type.**
   Payment types were grouped into three categories: `Cash`, `Credit Card`, and `other`. This reduced the number of rare categories and made the model more stable.

7. **Encoding cyclic datetime features.**
   Month, day, hour, and weekday are cyclic features. For example, hour 23 is close to hour 0, but a simple numerical representation does not show this relationship. Therefore, each cyclic feature was transformed using sine and cosine functions:

   $$x_{sin} = \sin\left(2\pi \frac{x}{period}\right), \quad x_{cos} = \cos\left(2\pi \frac{x}{period}\right)$$

   This transformation was applied to month, day, hour, and weekday.

8. **Filtering invalid labels.**
   Records with missing fare values or non-positive fare values were removed.

9. **Balanced sampling.**
   Since many rides had very small fares, the pipeline limited the number of records with fare less than or equal to $5. The remaining sample was filled with records having fare greater than $5. This reduced target bias toward very cheap trips.

10. **Train-test split.**
    The sampled data was split into training and test sets using an 80/20 ratio with a fixed random seed.

11. **Indexing categorical features.**
    `StringIndexer` was used to convert categorical values into category indices. Invalid or unseen values were kept using `handleInvalid = keep`.

12. **One-hot encoding.**
    `OneHotEncoder` was applied to indexed categorical features to convert them into sparse binary vectors.

13. **Imputing numerical values.**
    Missing numerical values were filled using median imputation with Spark ML `Imputer`.

14. **Assembling the final feature vector.**
    All imputed numerical features and encoded categorical features were combined into one `features` vector using `VectorAssembler`.

4.2 Final Feature Groups

The final model input consisted of:

- numerical features: `trip_seconds`, `trip_miles`, `trip_year`;
- boolean validity flags: `is_valid_end_timestamp`, `is_valid_duration`, `is_valid_amounts`;
- cyclic datetime features based on month, day, hour, and weekday;
- one-hot encoded categorical features for payment type, pickup community area, and dropoff community area.

This pipeline made the data suitable for distributed machine learning and ensured that the same preprocessing logic was applied to both training and test data.

5 ML Modeling

For model training, we used Spark ML regression algorithms. The task was to predict the `fare` value using the assembled `features` vector.

We trained and compared three models:

- Linear Regression;
- Random Forest Regressor;
- Decision Tree Regressor.

For each algorithm, we first trained a default model and then used 3-fold cross-validation for hyperparameter tuning. The evaluation metric used during cross-validation was RMSE. Final model quality was evaluated using RMSE, MAE, and R^2 on the test set.

5.1 Cross-Validation Setup

The cross-validation process used the following setup:

- number of folds: 3;
- random seed: 42;
- parallelism: 2;
- evaluator: regression evaluator based on RMSE.

5.2 Hyperparameter Grids

The following hyperparameter grids were used during model selection.

5.2.1 Linear Regression Grid

For Linear Regression, the grid tested different regularization settings:

- `regParam`: controls the strength of regularization. In [0.1, 1.0];
- `elasticNetParam`: controls the balance between L1 and L2 regularization. In [0.5, 1.0]

The purpose of this grid was to check whether regularization improves generalization and reduces overfitting.

5.2.2 Random Forest Regressor Grid

For Random Forest Regressor, the grid tested tree complexity and ensemble structure:

- `maxDepth`: maximum depth of each tree, In [5, 10];
- `numTrees`: number of trees in the forest, In [20, 50];

The purpose of this grid was to find a balance between model accuracy and training time. Deeper trees can capture more complex patterns, while more trees usually improve stability but increase computation cost.

5.2.3 Decision Tree Regressor Grid

For Decision Tree Regressor, the grid tested the complexity of a single tree:

- `maxDepth`: maximum depth of the tree, In [5, 10];
- `minInstancesPerNode`: minimum number of records required in each node, In [1, 5];

The purpose of this grid was to avoid both underfitting and overfitting while keeping the model interpretable.

5.3 Evaluation Metrics

The models were evaluated using the following metrics:

- **RMSE** — Root Mean Squared Error, which penalizes large errors more strongly;
- **MAE** — Mean Absolute Error, which shows the average absolute prediction error;
- R^2 — coefficient of determination, which shows how much variance in the target variable is explained by the model.

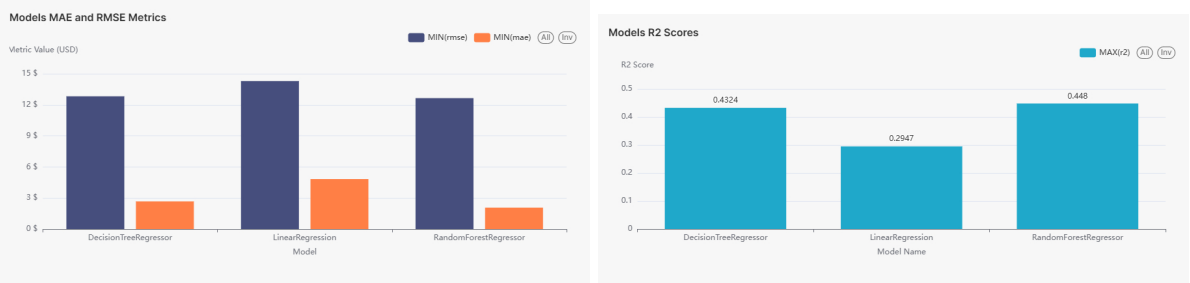

Figure 8: MAE, RMSE, and R2 Score

Model	RMSE	MAE	R^2
Linear Regression	14.2934	4.8289	0.2947
Random Forest Regressor	12.6458	2.0716	0.4480
Decision Tree Regressor	12.8226	2.6741	0.4324

Table 2: Final model performance on the test set

5.4 Model Results

The final test metrics are shown in Table 2.

The best model was **Random Forest Regressor**. It achieved the highest R^2 score of **0.4480**, which satisfies the business objective of achieving $R^2 > 0.40$. It also produced the lowest RMSE and MAE among all tested models.

The Decision Tree Regressor also passed the target R^2 threshold, but its MAE and RMSE were worse than those of the Random Forest model. Linear Regression performed the worst, which means that the relationship between features and fare is not purely linear.

5.5 Best Model Parameters

The best Random Forest Regressor was trained with the parameters shown in Table 3.

Parameter	Value
`bootstrap`	True
`cacheNodeIds`	False
`checkpointInterval`	10
`featureSubsetStrategy`	auto
`featuresCol`	features
`impurity`	variance
`labelCol`	label
`maxBins`	32
`maxDepth`	10
`maxMemoryInMB`	256
`minInfoGain`	0.0
`minInstancesPerNode`	1
`minWeightFractionPerNode`	0.0
`numTrees`	50
`predictionCol`	prediction
`seed`	42

Parameter	Value
`subsamplingRate`	1.0

Table 3: Best Random Forest Regressor parameters

6 Data Presentation

For convenient presentation of the results, we created a Superset dashboard. The dashboard contains visualizations related to exploratory data analysis and predictive data analysis.

The dashboard includes information about:

- fare distribution;
- trip distance and duration distribution;
- demand by hour, day, and month;
- payment type distribution;
- average fare by location;
- model performance comparison.

This dashboard helps users understand the data patterns and compare model quality in a clear visual form.

7 Conclusion

As a result of this project, we constructed a complete big data machine learning pipeline for taxi fare prediction. The pipeline includes data ingestion, storage in HDFS, exploratory data analysis, feature engineering, preprocessing, model training, hyperparameter tuning, evaluation, and dashboard presentation.

The main business objective was to build a model that predicts taxi fare and achieves $R^2 > 0.35$ on the test set. This objective was successfully achieved. The best model, Random Forest Regressor, reached:

- RMSE = 12.6458;
- MAE = 2.0716;
- $R^2 = 0.4480$.

This means that the model explains approximately 44.8% of the variance in taxi fares. The result is acceptable for the selected business objective, especially considering the high noise, outliers, missing values, and imbalance in the original dataset.

The project also showed that location, trip distance, trip duration, and time-based features are important for taxi fare prediction. Nonlinear models performed better than Linear Regression, which indicates that fare prediction depends on complex relationships between features.

8 Reflections on Own Work

During the project, we encountered several challenges.

- **Large dataset size.**
  The original dataset was very large, which led to long processing and training times. Some models required more than 20 minutes to train.
- **Hardware limitations.**
  During exploratory data analysis, it was difficult to process the full dataset locally because not all data could fit into memory. This required us to use distributed processing tools.
- **Target imbalance.**
  The fare distribution was highly biased toward low fare values, especially the 0–5 dollar range. This made the model more likely to predict low fares and reduced accuracy for rare expensive trips.
- **Outliers.**
  Some rides had very high fares. These records increased RMSE and made the regression task harder.
- **Overcomplicated cross-validation grids.**
  Initially, the cross-validation grids contained too many hyperparameter combinations. This significantly increased training time, especially for Random Forest and Decision Tree models.
- **Feature selection decisions.**
  Some columns had to be removed because of missing values or possible data leakage. For example, tips and trip total were strongly related to the target, so they were excluded from training.

Despite these challenges, the final solution achieved the required business objective. In future work, the model could be improved by using more advanced sampling strategies, additional geospatial features, outlier processing, and more efficient hyperparameter tuning.

9 Contributions

The team contribution is summarized in Table 4. Each percentage shows the approximate contribution of a team member to a particular project stage.

Project Task	Marsel Berheev ML Specialist	Makar Egorov Data Engineer	Rail Sharipov Data Scientist	Vlad Strelkov Tester and Technical Writer	Deliverables	Average Hours
Stage 1: Data Ingestion	0%	100%	0%	0%	Raw data in HDFS, source tables	8
Stage 2: EDA	60%	0%	40%	0%	EDA charts, insights, feature selection	16
Stage 3: PDA and ML Modeling	100%	0%	0%	0%	Trained models, metrics table, best model parameters	12
Stage 4: Report and Tests	10%	10%	10%	70%	Final report, tested pipeline, presentation materials	8

Table 4: Team contribution table